

Министр науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет информационных технологий и программирования

Лабораторная работа №4.

*Анализ и моделирование программной архитектуры и архитектуры данных
программного средства (средств) автоматизации ИТ-процессов во
взаимосвязи с функциональной и информационной архитектурой ИС.*

*Простой сокращатель ссылок (URL Shortener) на Python с использованием микрофреймворка
Flask и базы данных SQLite.*

Выполнили студенты группы № М3311

Агаев Эльдар
Ануфриев Константин
Сидякин Ярослав

Преподаватель:
Логачева Елизавета Дмитриевна

Санкт-Петербург
2026

Введение

Цель работы: построить модели и провести сопоставление функциональной, информационной, программной архитектур и архитектуры данных отдельного программного средства. В качестве программного средства рекомендуется выбрать небольшое open source решение, хотя можно использовать и проприетарное ПО, если есть доступ к кодовой базе и возможность представления его архитектуры. Если анализируемый продукт имеет достаточно богатый функционал, допустимо (по согласованию с преподавателем), раскрыть в работе только часть функций и их программной реализации.

Скрипт mini-shortener

В качестве объекта исследования выбрано open-source приложение для сокращения ссылок (URL Shortener), написанное на языке Python с использованием микрофреймворка Flask. Кодовая база проекта доступна в публичном репозитории GitHub: <https://github.com/shubham-kumar-9/URL-Shortener>. Данный проект оптимально подходит для анализа, так как обладает прозрачной архитектурой и реализует полный ИТ-цикл: от получения и валидации данных пользователя до взаимодействия с реляционной базой данных SQLite посредством объектно-реляционного отображения (ORM).

Стек: Python 3, Flask (веб-сервер), SQLite3 (БД), модуль string/random для генерации хэшей.

Часть 1. Описание типовых ИТ-процессов и требований

Описание процессов

Средство предназначено для автоматизации процесса трансформации длинных URL-адресов в короткие алиасы и маршрутизации пользователей по этим алиасам.

- **Сбор информации:** Пользователь отправляет длинный URL через POST-запрос.
- **Обработка:** Система валидирует URL и генерирует уникальный короткий идентификатор (токен из 6 символов).
- **Хранение:** Связка "Длинный URL ↔ Короткий токен" записывается в реляционную БД.
- **Предоставление/Передача:** При переходе по короткой ссылке система ищет токен в БД и отдает браузеру HTTP-статус 302 (Redirect) на оригинальный URL.

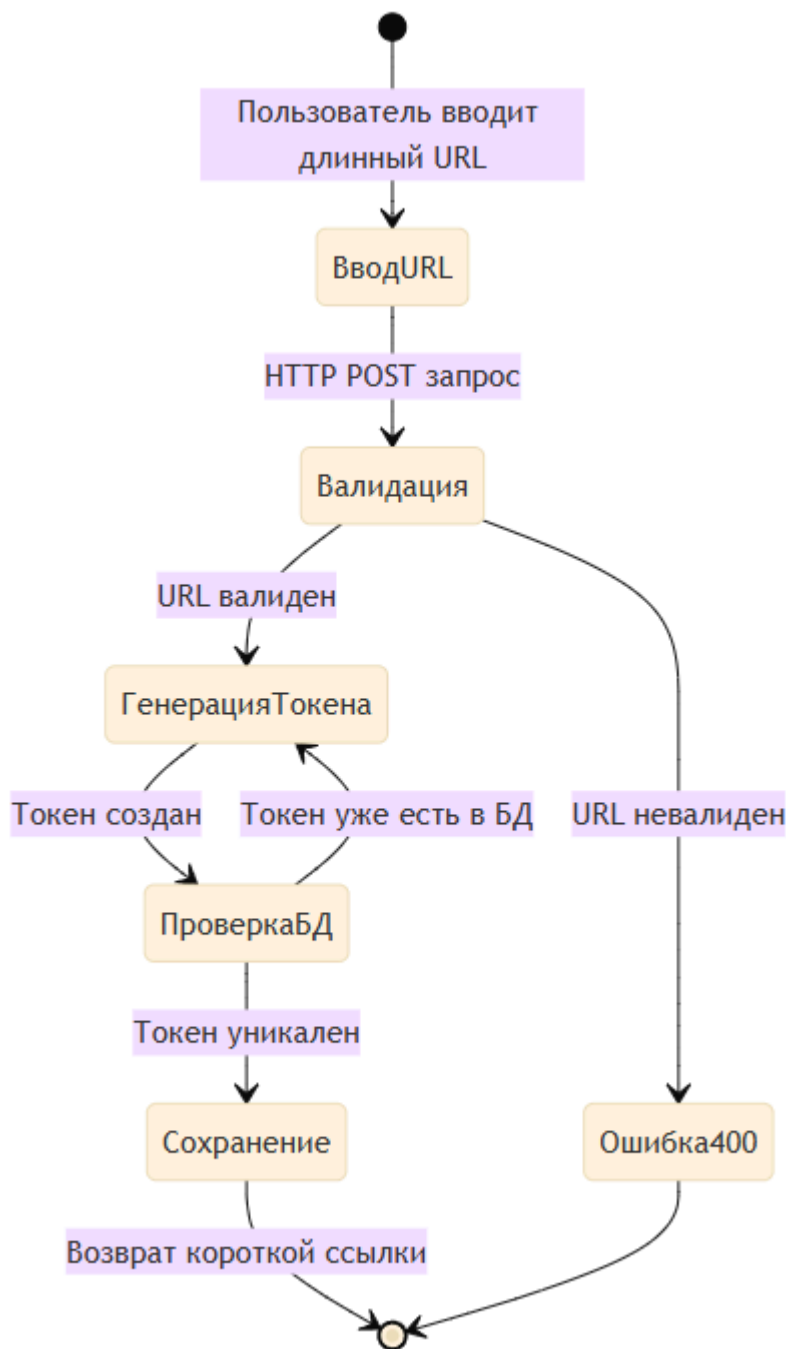
Информационные объекты

- Оригинальная ссылка (строка).
- Короткий токен (строка).

Требования

- **Функциональные:** Система должна генерировать уникальные токены; система должна осуществлять перенаправление; система должна подсчитывать количество переходов (кликов).
- **Нефункциональные:** Время отклика при редиректе не должно превышать 200 мс (показатель качества).

Activity Diagram



Часть 2. Структурированное описание программной архитектуры

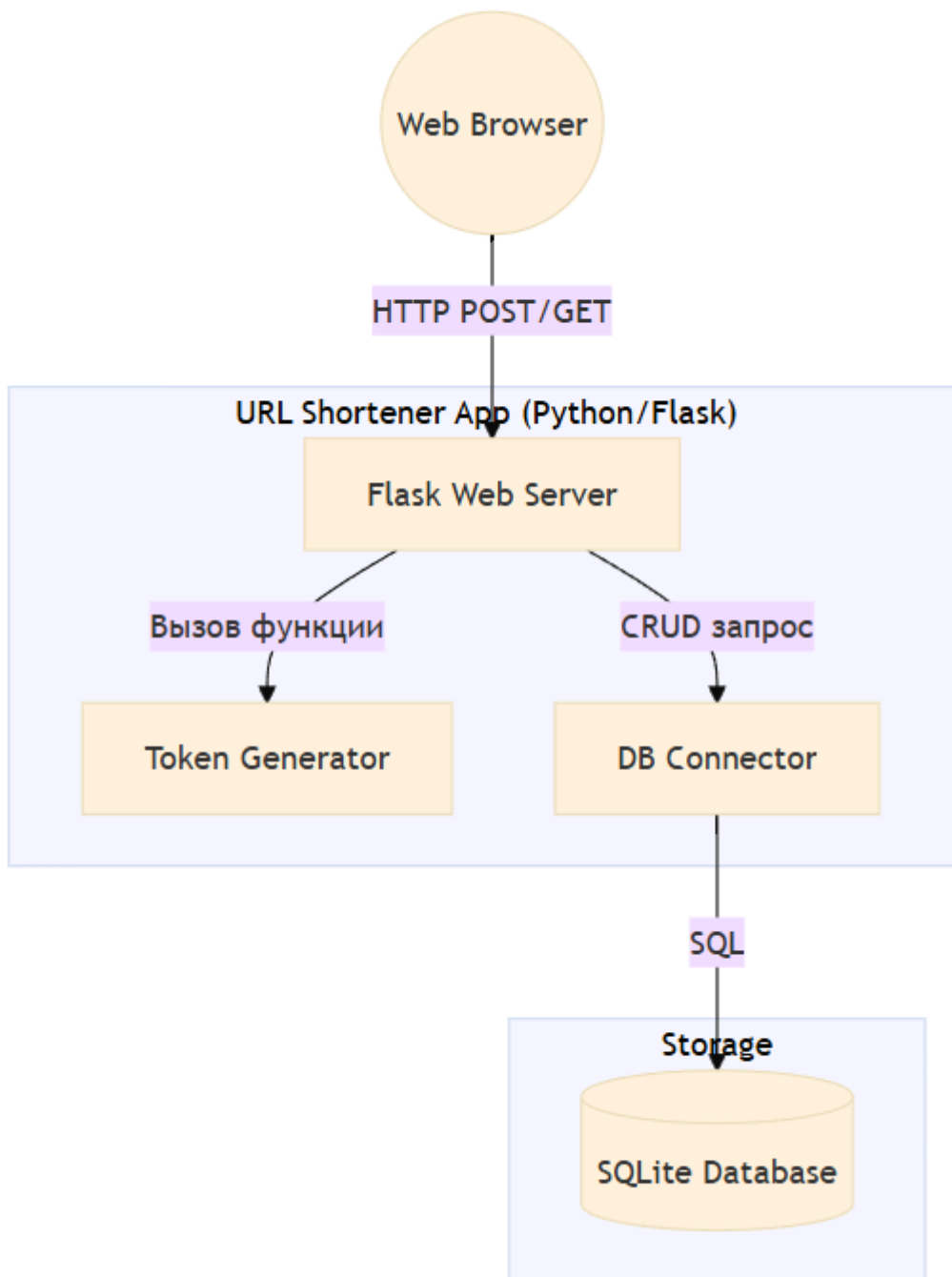
Программная архитектура (Компоненты)

Программная реализация системы базируется на паттерне MVC (Model-View-Controller). Точкой входа приложения выступает экземпляр класса Flask (в файле app.py), который обрабатывает входящие HTTP-запросы и распределяет их по функциям-контроллерам. За взаимодействие с базой данных отвечает класс URL (модель данных), который

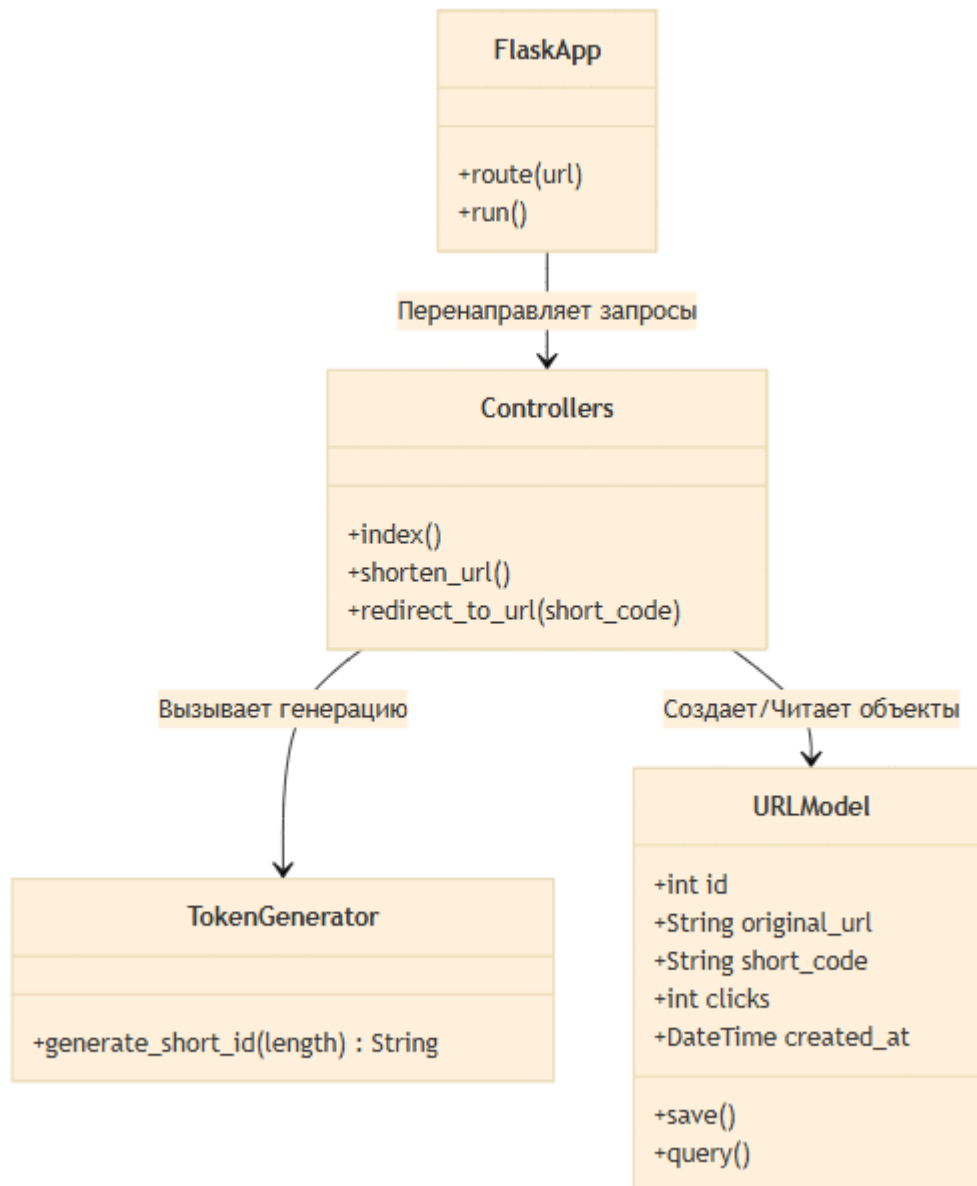
инкапсулирует в себе структуру таблицы urls и предоставляет встроенные методы для CRUD-операций. Логика генерации уникальных токенов вынесена в отдельную утилитарную функцию, использующую модули string и random для создания криптографически безопасных коротких алиасов.

- **Web Server (Flask App)** — точка входа, обрабатывает HTTP-запросы и маршрутизацию.
- **Token Generator** — вспомогательный модуль, использующий стандартные библиотеки Python (random, string) для создания алиасов.
- **Database Connector** — модуль взаимодействия с SQLite, выполняет SQL-запросы (CRUD операции).

Component Diagram



Как видно из представленной ниже диаграммы классов, программные компоненты имеют слабую связанность: веб-сервер делегирует работу с данными ORM-модели, что обеспечивает легкость модификации кода.

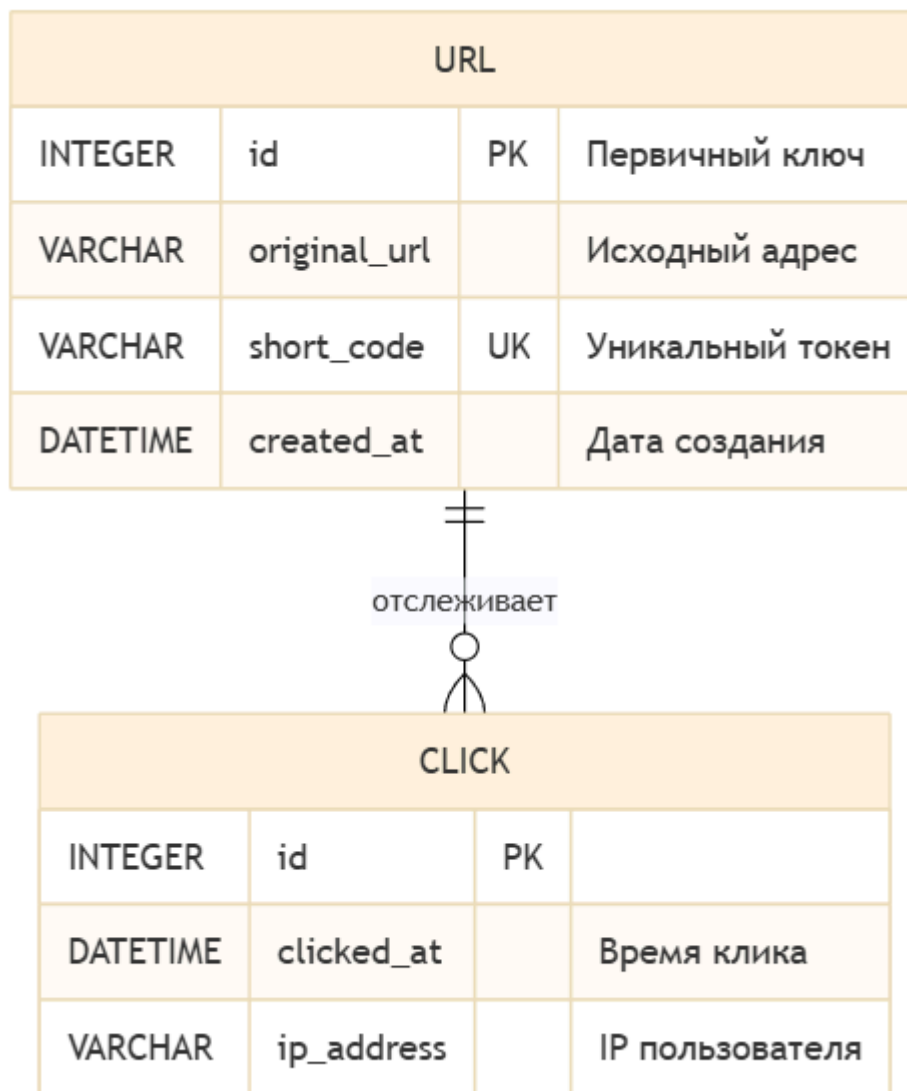


Архитектура данных

Данные хранятся в легковесной СУБД SQLite. Сложные пользовательские типы данных не используются. Таблица `urls` — основное хранилище.

Реляционная структура хранения информационных объектов представлена в виде ER-диаграммы (Entity-Relationship). Она описывает сущность URL-ссылки и типы хранимых данных.

Class/ER Diagram (Структура БД)



Часть 3. Сопоставление архитектур

Функциональная ↔ Программная архитектура

Функциональное требование	Программный компонент	Техническая реализация
Прием данных от пользователя	Web Server	Фреймворк Flask (методы <code>@app.route</code> , объект <code>request.form</code>).

Генерация алиаса	Token Generator	Модуль <code>random.choices</code> (Python Standard Library), выборка из <code>string.ascii_letters</code> + <code>string.digits</code> .
Сохранение ссылки	DB Connector	Библиотека <code>sqlite3</code> , выполнение <code>INSERT INTO urls....</code> . Используется совместно с <code>Web Server</code> .
Осуществление редиректа	<code>Web Server</code> + <code>DB Connector</code>	Компоненты работают одновременно: DB ищет ссылку (<code>SELECT</code>), а Flask выполняет функцию <code>redirect(url)</code> .

Информационная ↔ Архитектура данных

Информационный объект	Структура данных (Реализация)	Обеспечение целостности и уникальности
Оригинальная ссылка	Атрибут <code>original_url</code> (VARCHAR) в таблице <code>urls</code> .	Проверка на уровне приложения (регулярные выражения Flask) перед записью в БД.
Короткий токен	Атрибут <code>short_code</code> (VARCHAR) в таблице <code>urls</code> .	На уровне СУБД на поле наложено ограничение <code>UNIQUE</code> (уникальность). Целостность обеспечивается транзакционностью SQLite (ACID): если запись обрывается, токен не сохраняется.
Статистика (клики)	Атрибут <code>clicks</code> (INTEGER) в таблице <code>urls</code> .	Инкрементация счетчика <code>UPDATE urls SET clicks = clicks + 1</code> выполняется в рамках изолированной транзакции при каждом GET-запросе.

Выводы

В ходе выполнения лабораторной работы был проведен комплексный анализ программной архитектуры opensource-решения для сокращения URL-ссылок. Были построены и сопоставлены модели функциональной, информационной и программной архитектур в нотации UML.

Результаты сопоставления подтвердили, что выбранные программные компоненты (Flask-сервер, функции генерации) и структура данных (SQLite) полностью обеспечивают выполнение заданных ИТ-процессов. Информационная архитектура объекта эффективно отображена на реляционную модель, где целостность данных поддерживается на уровне СУБД (ACID-транзакции при обновлении счетчиков кликов), а уникальность идентификаторов гарантируется ограничением UNIQUE для поля короткого токена. Анализ показал, что текущая архитектура полностью удовлетворяет функциональным и нефункциональным требованиям к системе.